

Lab 24 - Metrics, Prometheus Exposition and the Observability Pipeline

Field	Value
Lab id	Lab 24
Day / module	Day 5 - Observability
Duration	45 minutes
Namespace	kcna-lab24
Maps to	LO6 Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting A6 Implement regular system reviews to monitor solution status and make modifications, according to an architecture management framework K7 Interactions among various IT components
Cluster	Single-node kind, Kubernetes v1.30+ (built and validated against v1.37.0)

How to read the expected-output blocks. Pod name suffixes, IP addresses,

AGE columns and timestamps vary at runtime. Metric values vary too -

counters depend on how long the generator has been running when you scrape.

Where a number is unpredictable it is shown as <counter>, <gauge> or

similar. What you match is the *structure*: metric names, # HELP/# TYPE

lines, label sets, and the relationships between series.

1. Scope and honesty statement - read first

This lab does not install kube-prometheus-stack. There is no Operator, no ServiceMonitor CRD, no Alertmanager, no Grafana and no node-exporter.

That is a deliberate teaching decision. Those components are correct for a production cluster and wrong for a 45-minute lab, because they hide the mechanism behind custom resources. You would learn to fill in a CRD without ever seeing a scrape.

Instead you get the smallest thing that is still genuinely Prometheus: one Pod, one scrape config you wrote yourself, two targets you can name, and a local TSDB you query with real PromQL over metric names you produced. Everything you observe here is real output from a real Prometheus server.

CNCF status, stated precisely - you will be asked this in the assessment:

Project	CNCF status	Role
Kubernetes	Graduated	Orchestrator
Prometheus	Graduated (the second project ever to graduate, 2018)	Metrics collection, storage, alerting
OpenTelemetry	Incubating - and the second-highest-velocity CNCF project after Kubernetes	Vendor-neutral generation and collection standard for traces, metrics and logs

Project	CNCF status	Role
Fluentd	Graduated	Log collection and forwarding
Jaeger	Graduated	Distributed tracing backend
Thanos, Cortex	Incubating	Long-term, horizontally scalable storage for Prometheus
Grafana	Not a CNCF project - open-source, Grafana Labs (AGPLv3)	Visualisation. Commonly taught alongside Prometheus; frequently and wrongly assumed to be CNCF
Loki	Not a CNCF project - Grafana Labs	Log aggregation

Sources: <<https://www.cncf.io/projects/>> and <<https://prometheus.io/>>.

2. Objective

By the end of this lab you will be able to:

1. Name the four observability signals - metrics, logs, traces, events (with profiles as an emerging fifth) - and say which one answers which question.
2. Explain pull vs push and give the concrete operational consequence of each in a Kubernetes cluster.
3. Read the Prometheus text exposition format line by line: # HELP, # TYPE, metric name, label set, sample value, and the _bucket/_sum/_count triple that makes up a histogram.
4. Distinguish counter, gauge and histogram and explain from first principles why rate() exists and why a raw counter is almost never what you want to graph.
5. Read a scrape config: job, targets, interval, timeout, and where relabel_configs sits relative to metric_relabel_configs.
6. Write PromQL against metric names you created yourself and interpret the results.
7. Diagnose a down scrape target using up and the /targets endpoint.
8. Place OpenTelemetry correctly in the pipeline relative to Prometheus.

3. Prerequisites

* A running single-node kind cluster.

```
kubectl version
```

```
Client Version: v1.37.0
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

* You are in the lab directory:

```
cd courseware/labs/lab-24-prometheus-observability
```

* Images used, all pinned: busybox:1.36, nginx:1.27-alpine, alpine:3.20, prom/prometheus:v2.54.1.

* Lab 23 is a soft prerequisite. You should already be able to say why metrics-server is not a monitoring system.

4. Scenario

Meridian Freight's post-incident review for INC-4417 produced one finding that nobody could argue with: **nobody could say when the error rate started rising.** The Depot Platform Squad had kubectl top - which, as you established in Lab 23, holds no history whatsoever - and a Slack thread.

Ticket DEP-1024:

*Instrument the Depot Portal so that "when did this start" is answerable.

Stand up a metrics pipeline in the lab cluster, expose gate throughput, queue

depth and scan latency, and demonstrate a query that would have detected

INC-4417 within one scrape interval.*

You are the platform engineer on DEP-1024.

5. Step-by-step procedure

Part 1 - The four signals, before you type anything

Signal	Answers	Cardinality	Retention	This course
Metrics	*How much? How often? How fast?*	Low - bounded by label combinations	Long (months) and cheap	This lab
Logs	Aggregated numbers over time			
Logs	*What exactly happened in this one request?*	High - one record per event	Short and expensive	Lab 23
Traces	Discrete timestamped records			
Traces	*Where did the time go across services?*	Very high; usually sampled	Short	Concept only
Events	Causally-linked spans			
Events	*What did the control plane decide?*	Low	1 hour by default	Lab 23
	Kubernetes-specific state changes			

The rule to remember. Metrics tell you **that** something is wrong and

when it started. Logs and traces tell you **why**. Alert on metrics;

investigate with logs and traces. Alerting on log volume is a common and

expensive mistake.

Part 2 - Deploy the scrape target

Step 2.1 - Create the namespace.

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab24 created
```

Step 2.2 - Load the generator script from data/ into a ConfigMap. Read the script first - it is the exposition format, written out by hand, with comments.

```
sed -n '25,60p' data/generate-metrics.sh
```

```
kubectl create configmap depot-metrics-generator \
  --from-file=generate-metrics.sh=data/generate-metrics.sh \
  -n kcna-lab24
```

```
configmap/depot-metrics-generator created
```

Step 2.3 - Apply the exporter's nginx config and the workload.

```
kubectl apply -f manifests/05-exporter-nginx-config.yaml
kubectl apply -f manifests/10-depot-metrics-sim.yaml
```

```
configmap/exporter-nginx-config created
deployment.apps/depot-metrics-sim created
service/depot-metrics-sim created
```

Step 2.4 - Wait for it.

```
kubectl rollout status deployment/depot-metrics-sim -n kcna-lab24 --timeout=120s
```

```
deployment "depot-metrics-sim" successfully rolled out
```

```
kubectl get pods -n kcna-lab24
```

NAME	READY	STATUS	RESTARTS	AGE
depot-metrics-sim-6b7d9f4c58-k2mvx	2/2	Running	0	35s

2/2 - the generator writing the file, and the exporter serving it.

Part 3 - Read the exposition format, line by line

Step 3.1 - Start the in-cluster client.

```
kubectl apply -f manifests/30-metrics-client.yaml
kubectl wait --for=condition=Ready pod/metrics-client -n kcna-lab24 --timeout=60s
```

```
pod/metrics-client created
pod/metrics-client condition met
```

Step 3.2 - Scrape the target by hand. This is *exactly* what Prometheus does - an HTTP GET, nothing more.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics
```

```
# HELP depot_portal_shipments_scanned_total Shipment barcodes scanned at the depot gate since process start.
# TYPE depot_portal_shipments_scanned_total counter
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} <counter>
# HELP depot_portal_shipments_rejected_total Shipments rejected at the gate since process start.
# TYPE depot_portal_shipments_rejected_total counter
depot_portal_shipments_rejected_total{depot="sg-tuas",lane="inbound",reason="manifest_not_indexed"} <counter>
# HELP depot_portal_http_requests_total HTTP requests handled by the depot portal.
# TYPE depot_portal_http_requests_total counter
depot_portal_http_requests_total{depot="sg-tuas",method="GET",status="200"} <counter>
depot_portal_http_requests_total{depot="sg-tuas",method="POST",status="502"} <counter>
# HELP depot_portal_queue_depth Shipments currently waiting in the gate queue.
# TYPE depot_portal_queue_depth gauge
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} <gauge, 12-51>
# HELP depot_portal_active_drivers Drivers currently checked in at the depot.
# TYPE depot_portal_active_drivers gauge
depot_portal_active_drivers{depot="sg-tuas"} <gauge, 4-12>
# HELP depot_portal_scan_latency_seconds Time taken to scan and validate one shipment.
# TYPE depot_portal_scan_latency_seconds histogram
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="0.5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="1"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="2.5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="5"} <cumulative>
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas",le="+Inf"} <cumulative>
depot_portal_scan_latency_seconds_sum{depot="sg-tuas"} <sum>
depot_portal_scan_latency_seconds_count{depot="sg-tuas"} <count>
# HELP depot_portal_build_info Build metadata. Always 1; the LABELS carry the information.
# TYPE depot_portal_build_info gauge
depot_portal_build_info{version="6.0.0",revision="a41f9c2",depot="sg-tuas"} 1
```

Step 3.3 - Anatomy of one line. Take this line apart:

depot_portal_shipments_scanned_total{depot="sg-tuas", lane="inbound"} 147		
metric name	label set	value

* Metric name - snake_case. The _total suffix is the strong convention for counters. Units belong in the name (seconds, bytes), never in a label.

* Label set - key/value pairs in braces. Each distinct combination is a separate time series. This is where cardinality explosions come from: put a user ID or a request ID in a label and you create one series per user.

* Value - a 64-bit float. There is no integer type.

* Timestamps are optional in the exposition and almost always omitted; Prometheus stamps the sample with its own scrape time.

Step 3.4 - # HELP and # TYPE. These are metadata comment lines, one pair per metric family, and they are not decoration:

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics \
  | grep '^#'
```

```
# HELP depot_portal_shipments_scanned_total Shipment barcodes scanned at the depot gate since
process start.
# TYPE depot_portal_shipments_scanned_total counter
# HELP depot_portal_shipments_rejected_total Shipments rejected at the gate since process
start.
# TYPE depot_portal_shipments_rejected_total counter
# HELP depot_portal_http_requests_total HTTP requests handled by the depot portal.
# TYPE depot_portal_http_requests_total counter
# HELP depot_portal_queue_depth Shipments currently waiting in the gate queue.
# TYPE depot_portal_queue_depth gauge
# HELP depot_portal_active_drivers Drivers currently checked in at the depot.
# TYPE depot_portal_active_drivers gauge
# HELP depot_portal_scan_latency_seconds Time taken to scan and validate one shipment.
# TYPE depot_portal_scan_latency_seconds histogram
# HELP depot_portal_build_info Build metadata. Always 1; the LABELS carry the information.
# TYPE depot_portal_build_info gauge
```

TYPE is what tells Prometheus (and you) whether rate() is legal on this series. # HELP is what the UI shows an engineer at 3 a.m.

Step 3.5 - Counter vs gauge, demonstrated. Scrape twice, 12 seconds apart, and compare.

```
kubectl exec -n kcna-lab24 metrics-client -- sh -c '
  URL=http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics
  echo "--- scrape 1"
  wget -qO- $URL | grep -E "^depot_portal_(shipments_scanned_total|queue_depth)"
  sleep 12
  echo "--- scrape 2"
  wget -qO- $URL | grep -E "^depot_portal_(shipments_scanned_total|queue_depth)"
  ,
```

```
--- scrape 1
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} 63
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} 47
--- scrape 2
depot_portal_shipments_scanned_total{depot="sg-tuas",lane="inbound"} 72
depot_portal_queue_depth{depot="sg-tuas",lane="inbound"} 26
```

The two exact numbers above will differ on your run - that is the point.

What must hold, every time:

- * the counter went UP (63 → 72). A counter never decreases while the process lives. Its absolute value is meaningless: it depends only on how long the process has been up.
- * the gauge moved in an arbitrary direction (47 → 26). A gauge is a current reading. Its absolute value *is* meaningful.

>

Record both pairs in data/exposition-worksheet.csv.

Step 3.6 - Why rate() must exist. "147 shipments scanned" is not an answer to any operational question, because it is a lifetime total. What you want is **shipments per second, now**. That derivative - computed over a time window, and automatically corrected for counter resets when a process restarts - is rate(). There is no way to get it from a single scrape; it is inherently a query over stored history. This is precisely the capability metrics-server does not have.

Step 3.7 - Read the histogram. A histogram is not one series, it is a family:

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics \
  | grep scan_latency
```

```
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas", le="0.5"} 60
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas", le="1"} 81
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas", le="2.5"} 93
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas", le="5"} 96
depot_portal_scan_latency_seconds_bucket{depot="sg-tuas", le="+Inf"} 97
depot_portal_scan_latency_seconds_sum{depot="sg-tuas"} 39
depot_portal_scan_latency_seconds_count{depot="sg-tuas"} 97
```

* le means less than or equal to. Buckets are cumulative: the le="1" bucket contains everything in le="0.5" too. Values must therefore be non-decreasing as le rises - check that in your own output.

* le="+Inf" always equals _count. If it does not, the exporter is broken.

* _sum / _count gives the mean. Means hide outliers, which is exactly why buckets exist: histogram_quantile() estimates p95/p99 from them.

* Quantile accuracy is bounded by your bucket boundaries. You cannot recover precision you did not define up front - the single most important practical consequence of choosing buckets.

Step 3.8 - The _info pattern. depot_portal_build_info is always 1. The *value* carries nothing; the labels carry version and revision. This lets you join build metadata onto other series in PromQL rather than duplicating a version label onto every metric (which would multiply your cardinality).

Step 3.9 - Confirm the Content-Type.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -S -qO /dev/null \
  http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics 2>&1 \
  | grep -i 'content-type'
```

```
Content-Type: text/plain; version=0.0.4; charset=utf-8
```

version=0.0.4 identifies the text exposition format version. Prometheus uses this header to select a parser.

Part 4 - Pull vs push, and the scrape config

Step 4.1 - The model. Prometheus pulls. It connects to each target on a schedule and does an HTTP GET. Targets do not send anything.

	Pull (Prometheus)	Push (StatsD, OTLP push, Graphite)
Who initiates	The monitoring system	The application
Target discovery	Required - service discovery or static config	Not required
Is the target up?	Free - a failed scrape *is* the signal (up == 0)	Needs a separate heartbeat; silence is ambiguous
Short-lived jobs	Awkward - needs a Pushgateway	Natural
Firewall direction	Monitoring must reach targets	Targets must reach monitoring
Overload behaviour	Monitoring controls the rate	Targets can flood the collector

The decisive advantage of pull, in one line: you get liveness detection for free. `up == 0` tells you a target is unreachable without the target having to do anything. You will prove this in Part 6.

Step 4.2 - Read the scrape config before you apply it.

```
cat data/prometheus-scrape.yml
```

Work through its anatomy:

Key	Meaning	Value here
<code>global.scrape_interval</code>	Default pull frequency; the resolution of every stored series	10s
<code>global.evaluation_interval</code>	How often rules are evaluated	30s
<code>global.external_labels</code>	Stamped on series leaving this server	cluster, env
<code>scrape_configs[].job_name</code>	Becomes the job label on every series from this job	prometheus, depot-portal
<code>static_configs[].targets</code>	host:port list. Production uses <code>kubernetes_sd_configs</code> instead	Service DNS name
<code>static_configs[].labels</code>	Extra labels stamped on every series from these targets	depot, service
<code>metrics_path</code>	Path to GET	/metrics (the default)
<code>scrape_timeout</code>	Must be $\leq$ <code>scrape_interval</code>	5s
<code>relabel_configs</code>	Runs before the scrape; rewrites the <code>*target's*</code> labels	Strips :9100 into instance
<code>metric_relabel_configs</code>	Runs after the scrape; can drop individual <code>*series*</code>	Not used here

Relabelling in outline. Both stages operate on label sets with

`source_labels` → `regex` → `target_label` → `replacement` → `action`.

`relabel_configs` decides `*what to scrape and how to label it*`;

`metric_relabel_configs` decides `*what to keep*`. The usual production use of

the second is `action: drop` on high-cardinality series that would otherwise

bloat the TSDB.

Step 4.3 - Create the config ConfigMap. The key must be `prometheus.yml` because that is the filename `--config.file` points at.

```
kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape.yml \
  -n kcna-lab24
```

```
configmap/prometheus-config created
```

Step 4.4 - Deploy Prometheus.

```
kubectl apply -f manifests/20-prometheus.yaml
kubectl rollout status deployment/prometheus -n kcna-lab24 --timeout=180s
```

```
deployment.apps/prometheus created
service/prometheus created
deployment "prometheus" successfully rolled out
```

Step 4.5 - Confirm it loaded your config and both targets are UP. Allow ~20 seconds for the first scrape cycle.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- 'http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/targets?state=active' \
  | tr ',' '\n' | grep -E '"job"|"health"|"scrapeUrl"'
```

```
"scrapeUrl": "http://localhost:9090/metrics"
"health": "up"
"job": "prometheus"
"scrapeUrl": "http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9100/metrics"
"health": "up"
"job": "depot-portal"
```

Two targets, both "health": "up".

Step 4.6 - Optionally open the web UI. Leave this running in a second terminal.

```
kubectl port-forward -n kcna-lab24 svc/prometheus 9090:9090
```

```
Forwarding from 127.0.0.1:9090 -> 9090
Forwarding from [::1]:9090 -> 9090
```

Browse <<http://localhost:9090/targets>> and <<http://localhost:9090/graph>>.

Part 5 - PromQL against metrics you produced

Every query below runs against the metric names you defined in `data/generate-metrics.sh`. Use the web UI, or the API from the client Pod. A helper to keep the commands short:

```
promql() {
  kubectl exec -n kcna-lab24 metrics-client -- \
    wget -qO- "http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/query?query=$1"
}
```

URL-encode special characters: { is %7B, } is %7D, " is %22,

[is %5B,] is %5D, a space is %20. The web UI needs none of this,

which is why it is easier for exploration.

Step 5.1 - Instant vector. Ask for the raw counter.

```
promql 'depot_portal_shipments_scanned_total'
```

```
{"status": "success", "data": {"resultType": "vector", "result": [{"metric": {"__name__":
"depot_portal_shipments_scanned_total", "cluster": "kcna-kind", "depot": "sg-tuas", "env": "lab",
"instance": "depot-metrics-sim.kcna-lab24.svc.cluster.local", "job": "depot-portal", "lane":
"inbound", "service": "depot-portal"}, "value": [<timestamp>, <counter>]}]}}
```

Look at the labels you did not write. job came from job_name.

instance was produced by your relabel_configs - note it has **no

:9100**, which proves the relabel rule fired. cluster and env came from

external_labels. depot, lane and service came from the exposition and

the static config. Prometheus composed all of it.

Step 5.2 - rate(). The query that actually answers "how busy is the gate?"

```
promql 'rate(depot_portal_shipments_scanned_total%5B1m%5D)'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "depot": "sg-tuas",
          "env": "lab",
          "cluster": "kcna-kind",
          "instance": "depot-metrics-sim.kcna-lab24.svc.cluster.local",
          "job": "depot-portal",
          "lane": "inbound",
          "service": "depot-portal"
        },
        "value": [
          <timestamp>,
          <per-second rate>
        ]
      }
    ]
  }
}
```

Note the missing __name__. rate() returns a *new* series that is no

longer the original metric, so the name is dropped.

>

Sanity-check the magnitude yourself. The generator adds between 7 and 11

to the counter every 10 seconds, so the true rate is between **0.7 and 1.1

per second**. If your result is in that band, your whole pipeline - exposition,

scrape, storage, query - is provably correct end to end. This is a prediction

you can verify, not a number to copy.

Step 5.3 - The [1m] window rule. rate() needs at least two samples in the window. Your scrape_interval is 10s, so [1m] gives about six - ample. Try a window narrower than the interval and watch it fail:

```
promql 'rate(depot_portal_shipments_scanned_total%5B5s%5D)'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": []
  }
}
```

An empty result, not an error. Rule of thumb: make the range at least 4× your scrape interval.

Step 5.4 - Aggregate away a label. Sum across HTTP status codes:

```
promql 'sum%20by%20(status)%20(rate(depot_portal_http_requests_total%5B2m%5D))'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "status": "200"
        },
        "value": [
          <ts>,
          <rate>
        ]
      },
      {
        "metric": {
          "status": "502"
        },
        "value": [
          <ts>,
          <rate>
        ]
      }
    ]
  }
}
```

Step 5.5 - The query that would have caught INC-4417. Error ratio:

```
promql 'sum(rate(depot_portal_http_requests_total%7Bstatus%3D%22502%22%7D%5B2m%5D))%20/%20sum(rate(depot_portal_http_requests_total%5B2m%5D))'
```

```
{ "status": "success", "data": { "resultType": "vector", "result": [ { "metric": {}, "value": [ <ts>, "<ratio between 0 and 1>" ] } ] } }
```

This is the shape of every good availability alert: a ratio of a filtered rate to a total rate, over the same window. It is dimensionless, so it does not need retuning when traffic grows. Alerting on absolute error *count* does. In the generator, errors accrue on one cycle in four while successes accrue every cycle, so expect a small ratio, well under 0.1.

Step 5.6 - histogram_quantile(). Estimated p95 scan latency:

```
promql 'histogram_quantile(0.95%2C%20sum%20by%20(le)%20(rate(depot_portal_scan_latency_seconds_bucket%5B5m%5D)))'
```

```
{ "status": "success", "data": { "resultType": "vector", "result": [ { "metric": {}, "value": [ <ts>, "<seconds>" ] } ] } }
```

Read the query inside out, because this is the most misread expression in PromQL. rate(..._bucket[5m]) converts each cumulative bucket counter into a per-second rate; sum by (le) aggregates across all series while **keeping the le label**, which the function requires; histogram_quantile(0.95, ...) then interpolates within the bucket where the 95th percentile falls. Dropping le from the by clause is the classic mistake and yields NaN.

Step 5.7 - Query the gauge, and compare.

```
promql 'depot_portal_queue_depth'
```

```
{ "status": "success", "data": { "resultType": "vector", "result": [ { "metric": { "__name__": "depot_portal_queue_depth", ... }, "value": [ <ts>, "<12-51>" ] } ] } }
```

No rate() here - and applying one would be a bug. rate() on a gauge is meaningless: it assumes monotonicity and treats every decrease as a counter reset. For gauges you use the value directly, or avg_over_time(), max_over_time(), delta().

Step 5.8 - Prometheus scraping itself. Compare your hand-built exposition against one produced by a mature client library:

```
promql 'prometheus_http_requests_total'
```

```
{ "status": "success", "data": { "resultType": "vector", "result": [ { "metric": { "__name__": "prometheus_http_requests_total", "code": "200", "handler": "/api/v1/query", "instance": "localhost:9090", "job": "prometheus", "component": "monitoring", "cluster": "kcna-kind", "env": "lab" }, "value": [ <ts>, "<counter>" ] }, ... ] } }
```

```
promql 'prometheus_tsdb_head_series'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "__name__": "prometheus_tsdb_head_series",
          ...
        },
        "value": [
          <ts>,
          <active series count>
        ]
      }
    ]
  }
}
```

prometheus_tsdb_head_series is the single most important capacity metric

for any Prometheus. It counts active series, which is what drives memory. A

cardinality explosion shows up here first, long before an OOMKill.

Step 5.9 - The free liveness metric.

```
promql 'up'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "__name__": "up",
          "component": "monitoring",
          "cluster": "kcna-kind",
          "env": "lab",
          "instance": "localhost:9090",
          "job": "prometheus"
        },
        "value": [
          <ts>,
          "1"
        ]
      },
      {
        "metric": {
          "__name__": "up",
          "cluster": "kcna-kind",
          "depot": "sg-tuas",
          "env": "lab",
          "instance": "depot-metrics-sim.kcna-lab24.svc.cluster.local",
          "job": "depot-portal",
          "service": "depot-portal"
        },
        "value": [
          <ts>,
          "1"
        ]
      }
    ]
  }
}
```

Both 1. up is synthesised by Prometheus, not exposed by the target - it is the free liveness signal that the pull model buys you.

Step 5.10 - Record your Part 5 results in data/exposition-worksheet.csv.

Part 6 - Failure injection (mandatory)

You will break the scrape config in the most common way it breaks: a correct, valid config that points at the wrong port.

Step 6.1 - Confirm the healthy baseline.

```
promql 'up%7Bjob%3D%22depot-portal%22%7D'
```

```
{
  "status": "success",
  "data": {
    ...
    "value": [
      <ts>,
      "1"
    ]
  }
}
```

Step 6.2 - Swap in the broken config. Note that this is a valid YAML file that Prometheus will load without complaint.

```
diff data/prometheus-scrape.yml data/prometheus-scrape-broken.yml | head -20
```

```
< # Job 2: the Depot Portal metrics simulator.
...
<       - depot-metrics-sim.kcna-lab24.svc.cluster.local:9100
---
>       # WRONG PORT - the Service listens on 9100.
>       - depot-metrics-sim.kcna-lab24.svc.cluster.local:9101
```

```
kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape-broken.yml \
  -n kcna-lab24 --dry-run=client -o yaml | kubectl apply -f -
```

```
Warning: resource configmaps/prometheus-config is missing the kubectl.kubernetes.io/last-applied-configuration annotation which was created by kubectl create --save-config or kubectl apply. The missing annotation will be applied automatically.
configmap/prometheus-config configured
```

Step 6.3 - Wait for the kubelet to propagate the ConfigMap into the Pod. This is not instant - projected ConfigMap volumes refresh on the kubelet sync period, typically up to ~60 seconds.

```
kubectl exec -n kcna-lab24 deploy/prometheus -- \
  grep -c 9101 /etc/prometheus/prometheus.yml
```

1

If this returns 0, the file has not refreshed yet. Wait 30 seconds and retry. Do not restart the Pod - you would lose the TSDB and the whole point of the hot reload.

Step 6.4 - Hot-reload Prometheus. `--web.enable-lifecycle` makes this possible without losing stored samples.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- --post-data='' \
  http://prometheus.kcna-lab24.svc.cluster.local:9090/-/reload
```

(no output - HTTP 200)

Step 6.5 - Wait ~20 seconds, then observe the failure. This is the whole lesson of the pull model.

```
promql 'up'
```

```
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "__name__": "up",
          "component": "monitoring",
          "instance": "localhost:9090",
          "job": "prometheus",
          ...
        },
        "value": [
          <ts>,
          "1"
        ]
      },
      {
        "metric": {
          "__name__": "up",
          "depot": "sg-tuas",
          "instance": "depot-metrics-sim.kcna-lab24.svc.cluster.local",
          "job": "depot-portal",
          ...
        },
        "value": [
          <ts>,
          "0"
        ]
      }
    ]
  }
}
```

`up{job="depot-portal"}` is now 0. Nobody had to report anything: the failure to connect *is* the signal.

Step 6.6 - Get the reason from the targets API.

```
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- 'http://prometheus.kcna-lab24.svc.cluster.local:9090/api/v1/targets?state=active' \
  | tr ',' '\n' | grep -E '"health"|"lastError"'
```

```
"health": "up"
"lastError": ""
"health": "down"
"lastError": "Get \"http://depot-metrics-sim.kcna-lab24.svc.cluster.local:9101/metrics\": dial tcp 10.96.x.x:9101: connect: connection refused"
```

connection refused - DNS resolved and the Service exists, but nothing listens on that port. Compare with the signatures in the Troubleshooting table.

Step 6.7 - The critical observation about stale data. Query the metric itself, then query it as a range:

```
promql 'depot_portal_queue_depth'
```

```
{"status":"success","data":{"resultType":"vector","result":[]}}
```

Empty - the series went STALE, it did not go to zero. When a scrape fails, Prometheus marks the series stale rather than inventing a value. This matters enormously for alerting: a rule written as `depot_portal_queue_depth > 40` will simply stop firing when the target dies, because there is no data to compare. An alert on `up == 0` is what catches that, and it is why every mature alert set has one.

Step 6.8 - Repair: restore the good config and reload.

```
kubectl create configmap prometheus-config \
  --from-file=prometheus.yml=data/prometheus-scrape.yml \
  -n kcna-lab24 --dry-run=client -o yaml | kubectl apply -f -
```

```
configmap/prometheus-config configured
```

Wait for propagation, then reload:

```
kubectl exec -n kcna-lab24 deploy/prometheus -- \
  grep -c 9100 /etc/prometheus/prometheus.yml
kubectl exec -n kcna-lab24 metrics-client -- \
  wget -qO- --post-data='' \
  http://prometheus.kcna-lab24.svc.cluster.local:9090/-/reload
```

```
1
```

Step 6.9 - Confirm recovery, and note that the history survived.

```
promql 'up%7Bjob%3D%22depot-portal%22%7D'
```

```
{"status":"success","data":{"... "value":[<ts>,"1"]}]}}
```

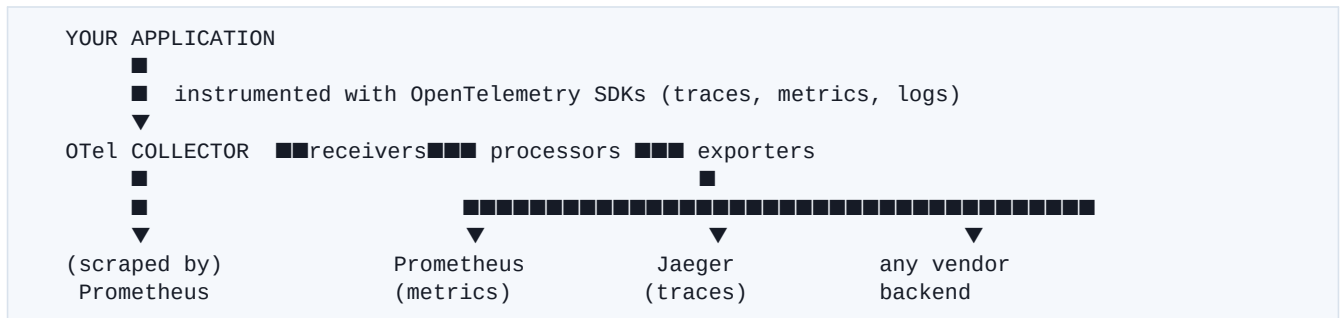
```
promql 'count_over_time(depot_portal_queue_depth%5B15m%5D)'
```

```
{"status":"success","data":{"resultType":"vector","result":[{"metric":{"..."}, "value":[<ts>,"<sample count>"]}]}}
```

The sample count spans the whole 15 minutes including the outage - the gap is visible as fewer samples than a perfect run would give. **You have just answered the INC-4417 question**: *when did it start, and how long did it last?* That is what the Depot Platform Squad could not do.

Part 7 - Where OpenTelemetry fits

Step 7.1 - Nothing to run. Fix the mental model, because this is examinable and widely muddled.



* OpenTelemetry is a generation and collection standard, not a backend. It defines the APIs, SDKs, semantic conventions and the OTLP wire protocol. It stores nothing and queries nothing. There is no "OpenTelemetry dashboard".

* Prometheus is a backend: it scrapes, stores in a TSDB, and serves PromQL.

* They are complementary, not competing. The common production shape is: instrument once with OTel SDKs, run the OTel Collector, and have it expose a Prometheus endpoint that Prometheus scrapes - or use Prometheus's native OTLP ingestion endpoint.

* Why it matters commercially: instrumenting with a vendor's proprietary agent locks your *code* to that vendor. Instrumenting with OTel means changing backends is a Collector config change, not a re-instrumentation project. That vendor-neutrality is the entire reason the project exists - and why it is the second-highest-velocity project in the CNCF.

Step 7.2 - Commit these three sentences to memory for the assessment:

1. *Prometheus is CNCF graduated; OpenTelemetry is CNCF incubating;

Grafana is not a CNCF project at all.*

2. *Prometheus pulls by default and synthesises up for free;

OTLP pushes.*

3. *rate() exists because counters are monotonic lifetime totals, and it automatically handles counter resets on process restart.*

6. Verification

```
bash verification/checks.sh
```

See verification/expected-output.md.

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix
Prometheus Pod CrashLoopBackOff, log shows opening storage failed: ... permission denied	The TSDB directory is not writable by uid 65534	kubectl logs -n kcna-lab24 deploy/prometheus	Ensure securityContext.fsGroup: 65534 is set on the Pod (present in the supplied manifest)
Prometheus Pod OOMKilled, RESTARTS climbing, reason: OOMKilled	Memory limit too small for the active series count	kubectl get pod -n kcna-lab24 -l app=prometheus -o jsonpath='{.items[0].status.containerStatuses[0].lastState.terminated.reason}'	Raise limits.memory; check prometheus_tsdb_head_series for a cardinality explosion
Target DOWN with connect: connection refused	Port is wrong, or nothing is listening	/api/v1/targets → lastError	Correct the port in the scrape config and POST /-/reload
Target DOWN with no such host	Service DNS name is wrong or the Service does not exist	kubectl exec metrics-client -- nslookup depot-metrics-sim.kcna-lab24.svc.cluster.local	Use <svc>.<ns>.svc.cluster.local; confirm the Service exists
Target DOWN with context deadline exceeded	Target is too slow; scrape_timeout exceeded	/api/v1/targets → lastError	Raise scrape_timeout (must stay <= scrape_interval), or make the exporter faster
Config edited but nothing changes	ConfigMap volume not yet refreshed, or no reload issued	kubectl exec deploy/prometheus -- cat /etc/prometheus/prometheus.yml	Wait for the kubelet sync (~60 s), then POST /-/reload
POST /-/reload returns 404	--web.enable-lifecycle is not set	kubectl get deploy prometheus -o jsonpath='{.spec.template.spec.containers[0].args}'	Add the flag (present in the supplied manifest)
rate() returns an empty result	Range window is shorter than ~2 scrape intervals, or the series is stale	Query the raw metric first	Widen the range to at least 4× scrape_interval
histogram_quantile() returns NaN	The le label was aggregated away	Inspect the inner sum by (...) clause	Always keep le: sum by (le) (rate(..._bucket[5m]))
Counters appear to jump backwards	More than one replica behind the scraped Service - each scrape hits a different Pod	kubectl get deploy depot-metrics-sim -o jsonpath='{.spec.replicas}'	Keep 1 replica for static scraping; in production discover Pods individually with kubernetes_sd_configs
Browser downloads /metrics instead of displaying it	Content-Type is application/octet-stream	wget -S -qO /dev/null <url>	Set default_type "text/plain; version=0.0.4" (done in 05-exporter-nginx-config.yaml)

8. Cleanup

Delete only this lab's namespace.

```
kubectl delete namespace kcna-lab24
```

```
namespace "kcna-lab24" deleted
```

```
kubectl get namespace kcna-lab24
```

```
Error from server (NotFound): namespaces "kcna-lab24" not found
```

Stop any kubectl port-forward left running with Ctrl-C.

This lab created no cluster-scoped objects and touched nothing in

kube-system. Deleting the namespace is a complete cleanup.

9. What you learned

- * The four signals divide by question, not by tooling. Metrics say **that** and **when**; logs and traces say **why**; Kubernetes Events say what the control plane decided. Alert on metrics, investigate with the rest.
- * The exposition format is just text over HTTP. # HELP, # TYPE, a metric name, a label set in braces, a float. You scraped it with wget before Prometheus ever touched it - there is no magic in the protocol.
- * Types determine legal operations. Counters only rise, so you must differentiate them with rate(). Gauges move freely, so you read them directly. Histograms are a **family** of cumulative `_bucket` series plus `_sum` and `_count`, and their quantile accuracy is fixed by bucket boundaries chosen in advance.
- * rate() exists because a lifetime total answers no operational question, and because it silently corrects for counter resets across restarts.
- * Pull gives you liveness for free. up is synthesised by Prometheus, not exposed by the target. You proved this by breaking one port number and watching up go to 0 with a precise lastError.
- * Failed scrapes produce staleness, not zeros. Threshold alerts silently stop firing when a target dies, which is why `up == 0` must be alerted separately.
- * Labels are the cardinality budget. Every distinct label combination is a separate time series; `prometheus_tsdb_head_series` is the metric that tells you when you have overspent.
- * `relabel_configs` runs before the scrape (what to scrape, how to label it); `metric_relabel_configs` runs after (what to keep).
- * OpenTelemetry generates and collects; Prometheus stores and queries. They are complementary. Prometheus and Fluentd and Jaeger are CNCF graduated; OpenTelemetry is incubating; Grafana is not a CNCF project.

Further reading

- * Prometheus - **Exposition formats**: <https://prometheus.io/docs/instrumenting/exposition_formats/>
- * Prometheus - **Metric types**: <https://prometheus.io/docs/concepts/metric_types/>
- * Prometheus - **Querying basics** and **Query functions**: <<https://prometheus.io/docs/prometheus/latest/querying/basics/>>
- * Prometheus - **Configuration → scrape_config and relabel_config**: <<https://prometheus.io/docs/prometheus/latest/configuration/configuration/>>
- * Prometheus - **Instrumentation and naming best practices**: <<https://prometheus.io/docs/practices/naming/>>
- * OpenTelemetry - **What is OpenTelemetry?**: <<https://opentelemetry.io/docs/what-is-opentelemetry/>>
- * CNCF - graduated and incubating project list: <<https://www.cncf.io/projects/>>
- * CNCF - KCNA curriculum, **Cloud Native Architecture → Observability**: <<https://github.com/cncf/curriculum>>